UNIVERSITY OF SAINT JOSEPH MBARARA (USJM)

Faculty of Education and Sciences

Bachelor of Science in Information Technology, Year 3

Mid-Semester One Examinations, Academic Year 2026/2027

ANDROID PROGRAMMING: TAKE-HOME EXAMINATION

Mid-Semester Stage: Project Report (Design Artifacts and Working Prototype)

USJM SMART LIBRARY MOBILE

An Android app that lets students and staff find, reserve and borrow library books without queuing, and gives librarians and administrators a mobile circulation desk

Item	Details
Student name	Tumusiime Henry
Registration number	2024-BSIT-007-F
Course	Android Programming (Year 3)
Course code	Course code
Course lecturer	Lecturer's name
Git repository	https://github.com/HENRIKYVYBZ/2024-BSIT-007-F-Tumusiime-Henry-AndroidExam
Date of submission	Date

Contents

If the list above is empty, right-click it in Word and choose Update Field → Update entire table. Delete this note afterwards.

How This Report Meets the Mid-Semester Requirements

The examination asks for six pieces of evidence at the mid-semester stage. The table shows where each one is in this report.

Requirement (Section 3 of the examination)	Where to find it	Status
Problem statement: the problem, its target users and why it matters (one page)	Section 1	Done
Requirements specification: functional and non-functional requirements	Section 3	Done: 28 functional, 10 non-functional
System design: use case diagram, at least one activity or sequence diagram, wireframes	Section 4 (Figures 1 to 9)	Done: 1 use case, 3 activity, 1 sequence diagram, 15 wireframes
Working prototype: at least two core screens, basic navigation, at least one functioning feature	Section 5	Done: 17 screens, working forms that save data, data shown from a local SQLite database
Version control evidence: a Git repository with incremental commits	Section 6	Done: incremental commits on GitHub
Short demo: a 3 to 5 minute recording or live walkthrough	Section 8 (demo script)	Add the link to your recording

The rubric for this stage awards marks for the problem definition and requirements (5), system design (5), prototype functionality (10), version control practice (5), and the demo and communication (5).

1. Problem Statement

1.1 Background

The University of Saint Joseph Mbarara (USJM) library serves students, lecturers and other staff across all faculties. Cataloguing and circulation are managed by a library system, but almost every interaction still happens in person at the circulation desk: readers ask whether a book is in and where it is shelved, queue to borrow it, and come back to return it. Librarians record loans and chase overdue books largely by hand.

1.2 The problem

- **Long queues at the circulation desk**, especially before lectures and during examinations, because searching, checking availability, borrowing and returning are all done at one desk.
- **No way to know or hold a book in advance.** A reader may walk to the library only to find every copy is out, and nothing stops the last copy being taken by someone else on the way.
- **Low visibility of the collection.** Many students do not know which books the library holds for their course units, or that the catalogue can be searched at all.
- **Manual work for librarians.** Pickups, loans, due dates and overdue reminders are tracked on paper or in a desktop system that librarians cannot carry to the shelves or the express counter.
- **Weak account control.** Without clear roles it is hard to give lecturers longer loans, give librarians desk rights, or stop a student from posing as staff.

As a result readers lose study time, books sit unused on shelves while others wait for them, and librarians spend much of the day answering the same availability questions.

1.3 Target users

User	Needs from the app
Students (primary)	Find a book from their phone, see if a copy is on the shelf, reserve it and collect it without queuing; get reminded to collect and to return
Teaching staff (lecturers)	The same as students, with longer loans and more reservations for teaching
Non-teaching staff	The same as students, with staff loan periods
Librarians	A mobile circulation desk: today's pickups, issue and return books, lend to walk-in readers, keep the catalogue up to date, remind borrowers
Administrators	Approve staff accounts, give the librarian role, suspend misused accounts

1.4 Why it matters

Nearly every student carries an Android smartphone, so putting the catalogue in their pocket shows them the library's resources at the moment they need them. Reserving ahead turns a long desk transaction into a short pickup at an express counter, shortening the queue for everyone. Librarians gain an accurate, live picture of holds and loans, and fewer books go missing or stay overdue.

1.5 Proposed solution

USJM Smart Library Mobile is a native Android application written in Java. Readers sign in, search a live catalogue, scan a book's QR label, reserve a copy for a pickup day and follow their reservations and loans. Librarians run the circulation desk from the same app, and administrators manage accounts and roles. Shared data lives in Cloud Firestore, protected by security rules, with an SQLite copy of the catalogue on every phone for fast, offline browsing. Librarians can also import the catalogue from my final-year project, the USJM Smart Library Management System (a Django web system), through its REST API. The course instructor approved the problem before development began.

2. Objectives and Scope

2.1 Objectives

General objective: to reduce queuing time and improve access to library resources at USJM by giving every library user a mobile app suited to their role.

Specific objectives:

1. To provide secure accounts with roles (student, teaching staff, non-teaching staff, librarian, administrator), password reset and two-step account recovery.
2. To display the library catalogue with live search, real-time availability and QR-code look-up.
3. To let readers reserve books through a validated form, with limits that depend on their role, and follow their reservations and loans.
4. To give librarians a mobile circulation desk for pickups, loans, returns, walk-in lending and catalogue maintenance.
5. To store data persistently and safely (SQLite on the phone, Cloud Firestore shared), and to test the app with unit and manual tests.

2.2 Scope

In scope at the mid-semester stage: every feature listed as Done in Section 3. **Planned for the final stage:** lecturer book recommendations and course reading lists, administrator usage statistics, a full test plan and a signed release APK. **Out of scope:** paying fines, inter-library loans and the cataloguing of new acquisitions, which stay in the web system.

3. Requirements Specification

3.1 Functional requirements

Priorities follow the MoSCoW method (Must, Should, Could). "Reader" means a student, teaching staff or non-teaching staff member.

ID	The system shall…	Actor	Priority	Status
Accounts				
FR1	let a person create an account by choosing their role (student, teaching staff, non-teaching staff), and entering their name, registration number or staff ID, email, password and a recovery PIN, each validated	Reader	Must	Done
FR2	let a person sign in with email and password, stay signed in on the phone, and sign out after confirming	All	Must	Done
FR3	send a password-reset link to the person's email; the link opens a screen (in the app, or a web page) where the new password is typed twice and saved only when both match	All	Must	Done
FR4	let a person who forgot their email find it in two steps (ID number, then recovery PIN), locking for 15 minutes after 5 wrong tries	All	Should	Done
FR5	keep staff accounts waiting for an administrator's approval, giving them student limits until then	Admin	Must	Done
FR6	show each person a colour-coded role badge and only the tools their role allows	All	Should	Done
Catalogue				
FR7	list the catalogue with title, author, subject, shelf and copies available, kept live across all phones	All	Must	Done
FR8	filter the catalogue by title, author or subject as the user types	All	Must	Done
FR9	show a book's details: description, year, shelf location and availability	All	Must	Done
FR10	open a book's details by scanning the QR label on the book	All	Could	Done
Reservations and loans				
FR11	let a reader reserve an available book for a pickup date, validating every field	Reader	Must	Done
FR12	refuse a reservation when no copy is left, when the reader already has the book, or when they reach their role's limit (students 3 holds and 7 days ahead; lecturers 10 and 14 days); the library is closed on Sundays	Reader	Must	Done
FR13	never give the last copy to two people at the same time	Reader	Must	Done
FR14	list the reader's reservations and loans live, with their status and due dates (red when overdue)	Reader	Must	Done
FR15	let a reader cancel a waiting reservation, returning the copy to the shelf	Reader	Should	Done
FR16	expire holds not collected by their pickup date and return the copy	System	Should	Done
FR17	remind the reader at 08:00 on the pickup day, and remind borrowers when a book is due tomorrow, due today or overdue	System	Should	Done
Circulation desk				

ID	The system shall…	Actor	Priority	Status
FR18	show the librarian tabs for Pickups (today flagged), On loan, Overdue and Stock (books with 0 or 1 copy left)	Librarian	Must	Done
FR19	let the librarian mark a hold Collected (a loan starts: 14 days for students, 30 for staff) or No-show, and a loan Returned	Librarian	Must	Done
FR20	let the librarian lend a book to a walk-in reader found by registration number or staff ID	Librarian	Should	Done
FR21	let the librarian add, edit and remove books, never reducing copies below those reserved or on loan; show each book's QR label	Librarian	Should	Done
FR22	let the librarian publish the sample catalogue or import the catalogue from the web system's REST API	Librarian	Should	Done
FR23	let the librarian remind an overdue borrower, who is notified on their phone	Librarian	Could	Done
Administration				
FR24	let an administrator approve or reject staff accounts, change anyone's role and suspend or restore accounts	Admin	Must	Done
Final stage				
FR25	let lecturers recommend books for the library to buy, and librarians mark them Ordered or Declined	Lecturer	Should	Final stage
FR26	let lecturers publish course reading lists that all readers can open	Lecturer	Could	Final stage
FR27	show administrators usage statistics (users by role, reservations by status, most reserved books)	Admin	Could	Final stage
FR28	search the catalogue by voice or barcode on the book's cover	Reader	Could	Final stage

3.2 Non-functional requirements

ID	Category	Requirement	How the design meets it
NFR1	Usability	A first-time reader can find and reserve a book in under 2 minutes without help.	Short flow, plain language, input hints, forms pre-filled from the profile, errors shown beside the field.
NFR2	Performance	The catalogue opens and search results update in under 1 second.	Search runs on the SQLite copy on the phone; list rows are reused (ViewHolder pattern).
NFR3	Data integrity	A copy is never given to two people; counts never go below 0 or above the total.	Firestore transactions; checks in the security rules; CHECK constraints in SQLite.
NFR4	Offline use	The catalogue and book details can be browsed without internet.	SQLite cache refreshed by a live listener; Firestore keeps its own offline copy.
NFR5	Security	Nobody can give themselves a role, approve themselves, read other people's loans or change copy counts without a real reservation.	Firebase Authentication; Firestore security rules check the role, owner and every changed field.

ID	Category	Requirement	How the design meets it
NFR6	Privacy	Passwords and recovery PINs are never stored on the phone or in the database.	Firebase stores passwords; only a SHA-256 fingerprint of ID number + PIN is saved.
NFR7	Reliability	Reminders still arrive after the phone restarts or without internet.	AlarmManager alarms, saved and restored by a boot receiver.
NFR8	Compatibility	Runs on Android 7.0 (API 24) and newer, on 5 to 7 inch phones, and keeps input when the screen rotates.	minSdk 24, flexible layouts in ScrollViews, saved instance state.
NFR9	Maintainability	Rules, data and screens can change independently.	Layers: screens, plain-Java rules (unit tested), data classes; one file per screen.
NFR10	Accessibility	Touch targets at least 48dp; text readable in sunlight; colour never the only signal.	Navy and white colour scheme with gold accents; role badges carry a text label as well as a colour.

4. System Design

4.1 Use case diagram

Figure 1 shows who uses the app and for what. Students, teaching staff and non-teaching staff are all kinds of **Reader** (generalisation), so they share the reader use cases. An **Administrator** is a special librarian and can also run the desk. Firebase (authentication and the shared database) and the library web system are external actors.

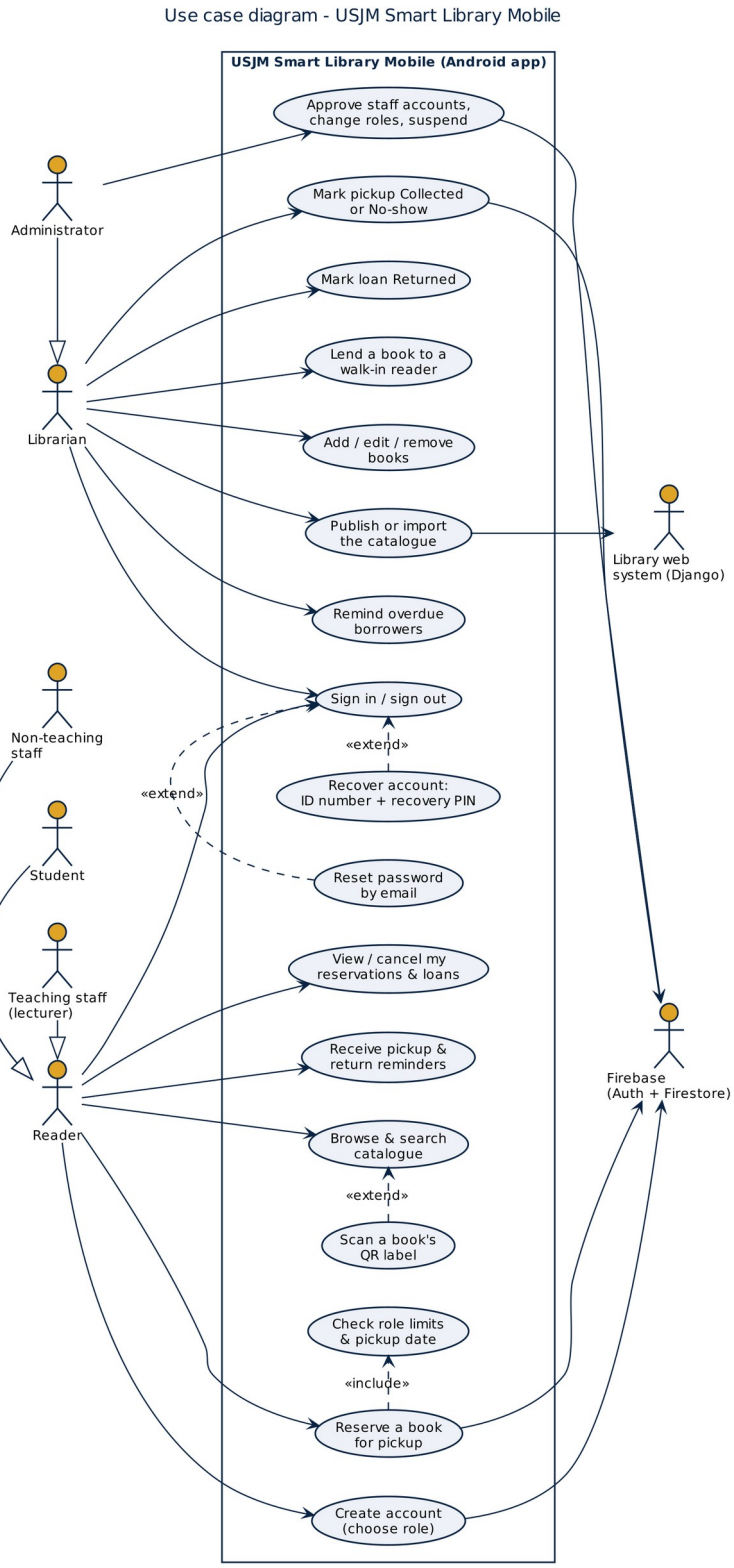


Figure 1: Use case diagram

Use case specification: Reserve a book for pickup

Element	Description
Actor	Reader (student, teaching staff or non-teaching staff)
Pre-condition	The reader is signed in and has opened a book that has at least one copy on the shelf.

Element	Description
Main flow	1. The reader taps "Reserve for pickup". 2. The app shows the form, pre-filled from the reader's profile, with a calendar limited by their role. 3. The reader confirms their name and ID number, picks a date and taps "Confirm reservation". 4. The app validates the fields and checks the role limits in Firestore. 5. A Firestore transaction takes one copy and saves the hold as ACTIVE. 6. The app schedules the pickup reminder and shows the pickup date, shelf and reservation code.
Alternative flows	4a. A field is invalid: the error is shown next to it and the reader corrects it. 4b. A rule is broken (duplicate, hold limit): the reason is shown and nothing is saved. 5a. Someone took the last copy a moment earlier: "All copies are out", nothing is saved.
Post-condition	An ACTIVE hold exists, the book has one copy fewer, and the librarian sees it under Pickups.

Use case specification: Mark a pickup Collected

Element	Description
Actor	Librarian (or administrator)
Pre-condition	The librarian is signed in; the reader has an ACTIVE hold and is at the desk.
Main flow	1. The librarian opens the circulation desk; today's pickups are flagged TODAY. 2. The librarian finds the hold (or scans the book's QR label to show only that book). 3. The librarian taps "Collected". 4. A transaction checks the hold is still ACTIVE and changes it to COLLECTED with a due date.
Alternative flows	3a. The reader did not come: "No-show" puts the copy back on the shelf. 4a. Another librarian already changed it: "This reservation was already updated".
Post-condition	The loan appears under On loan, and in the reader's My reservations with its due date.

4.2 Activity diagrams

Figure 2 shows how a person gets into the app: an existing session skips the sign-in screen; otherwise they sign in, create an account, recover a forgotten email in two steps, or reset a forgotten password. After sign-in the role decides where they land.

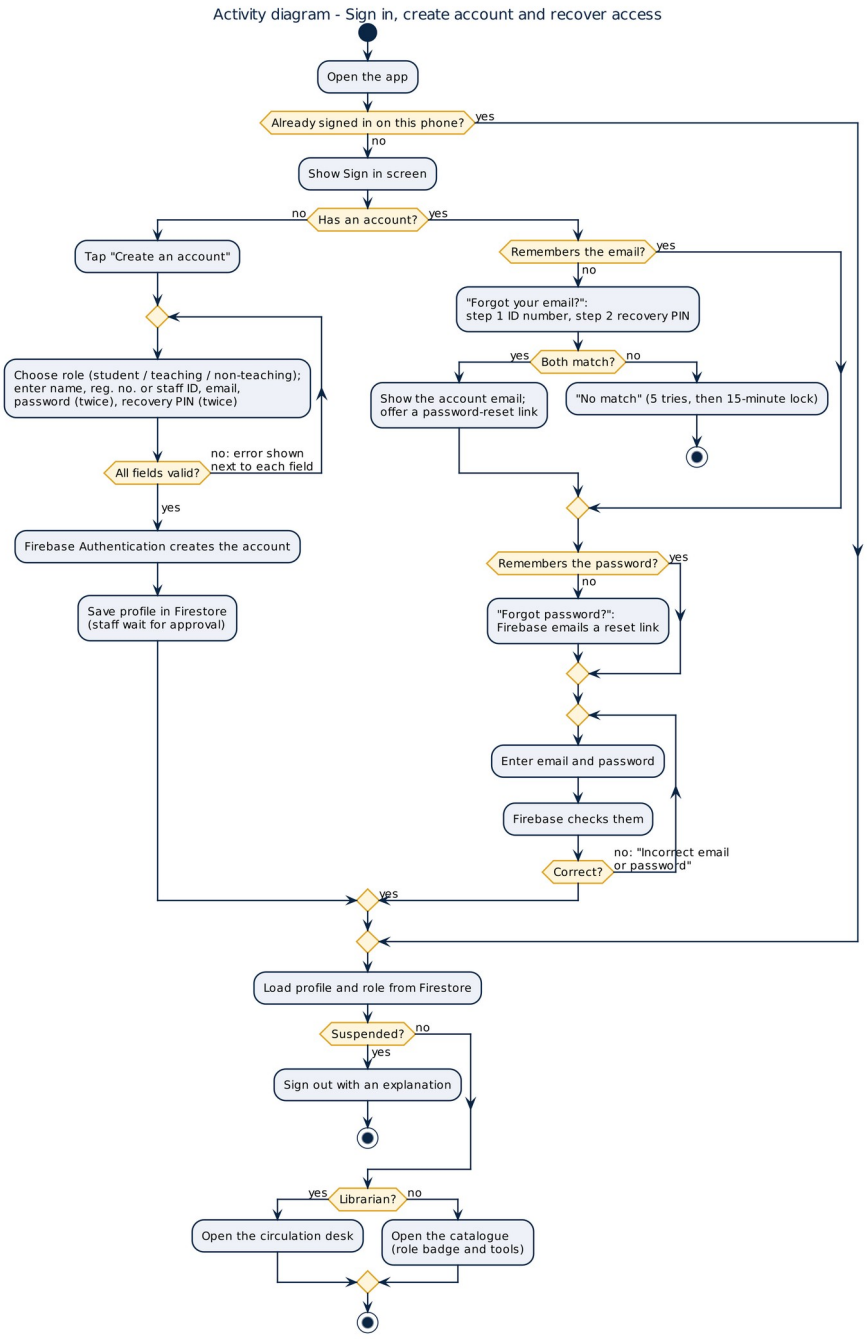


Figure 2: Activity diagram, sign in, create account and recover access

Figure 3 follows one book from reservation to return, across three swimlanes: the reader, the app and the librarian. It includes the validation loop and the transaction that refuses a copy someone else has just taken.

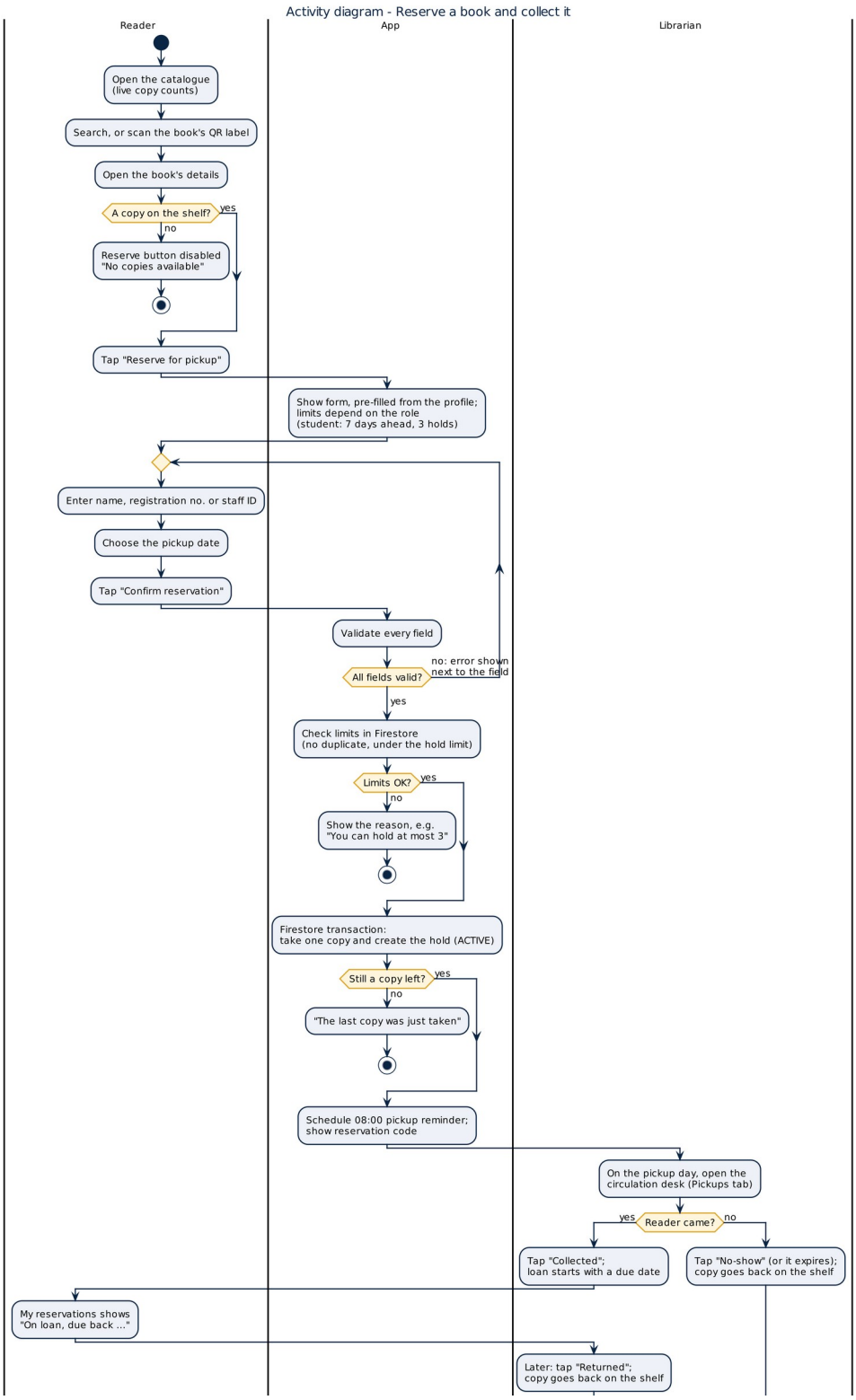


Figure 3: Activity diagram, reserve a book and collect it

Figure 4 shows the librarian's work at the desk: each reader at the counter is a pickup, a walk-in loan, a return or a stock task, and every change reaches all phones at once.

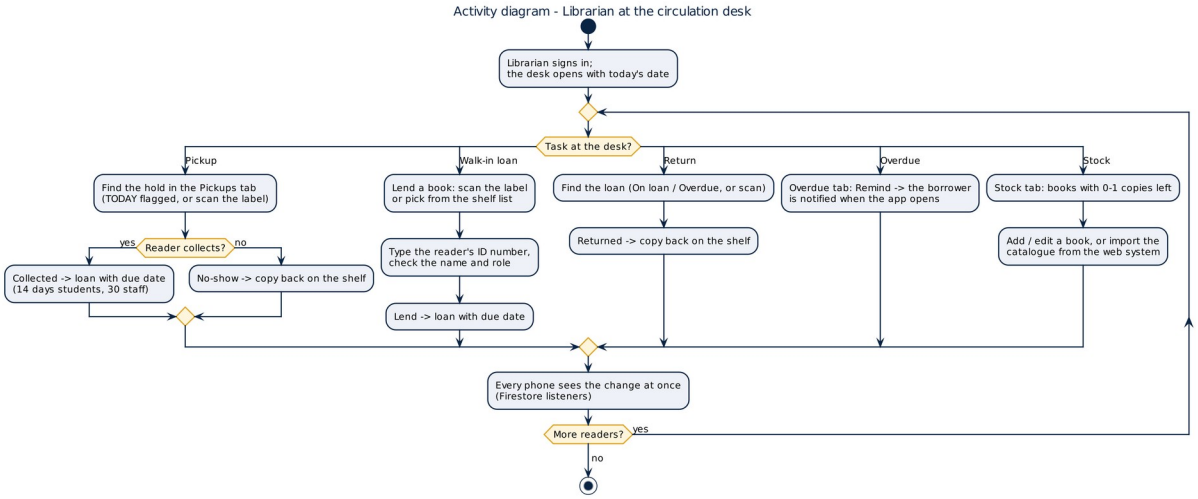


Figure 4: Activity diagram, the circulation desk

4.3 Sequence diagram

Figure 5 shows how the classes work together when a reader confirms a reservation. The screen asks Validator and Roles to check the input, then CloudLibrary checks the reader's limits and runs a Firestore transaction that takes the copy and saves the hold in one step. The security rules check the request on the server.

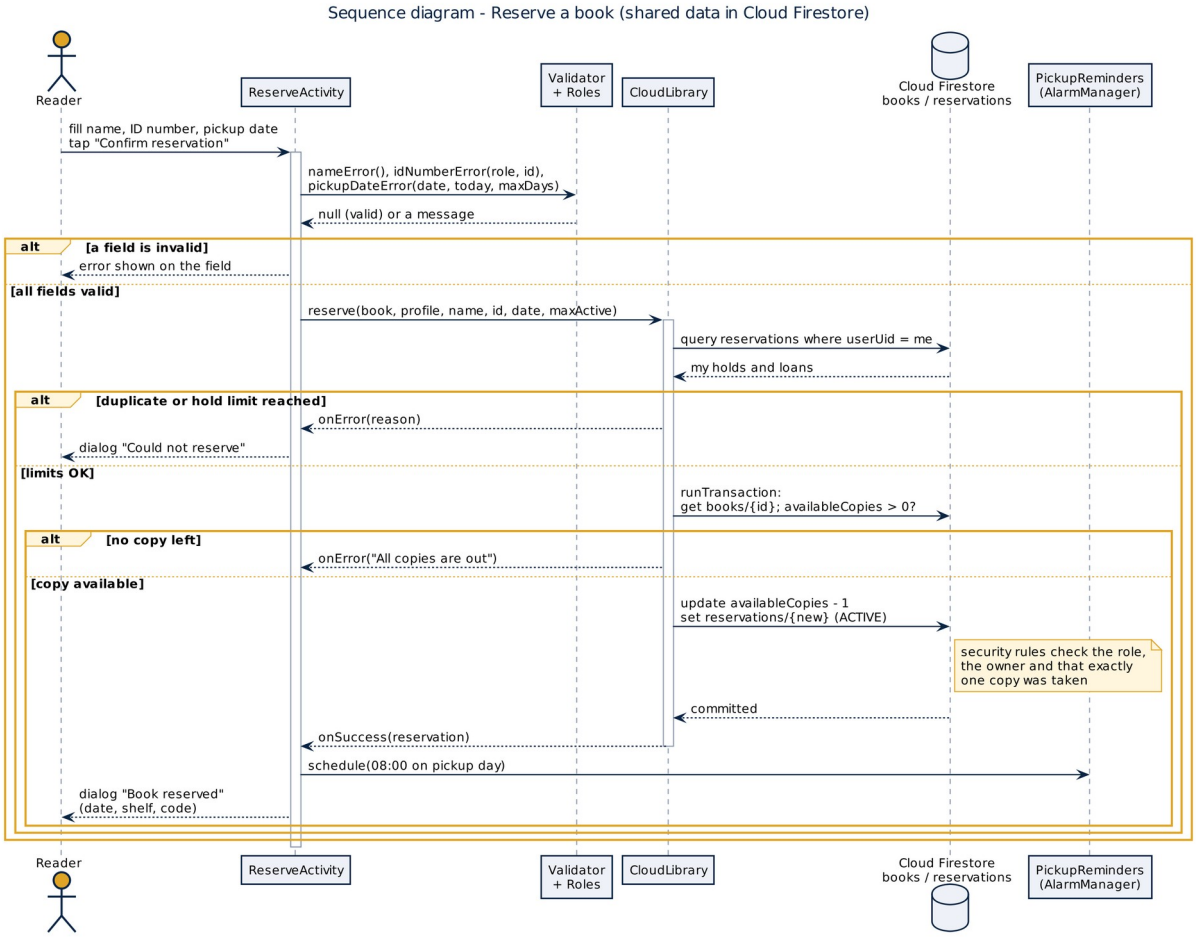


Figure 5: Sequence diagram, reserve a book

4.4 Architecture

The app is layered. Screens never talk to the database directly: they call the data classes, and all business rules (limits, dates, validation, recovery keys) are in plain Java classes that are unit tested on the computer.

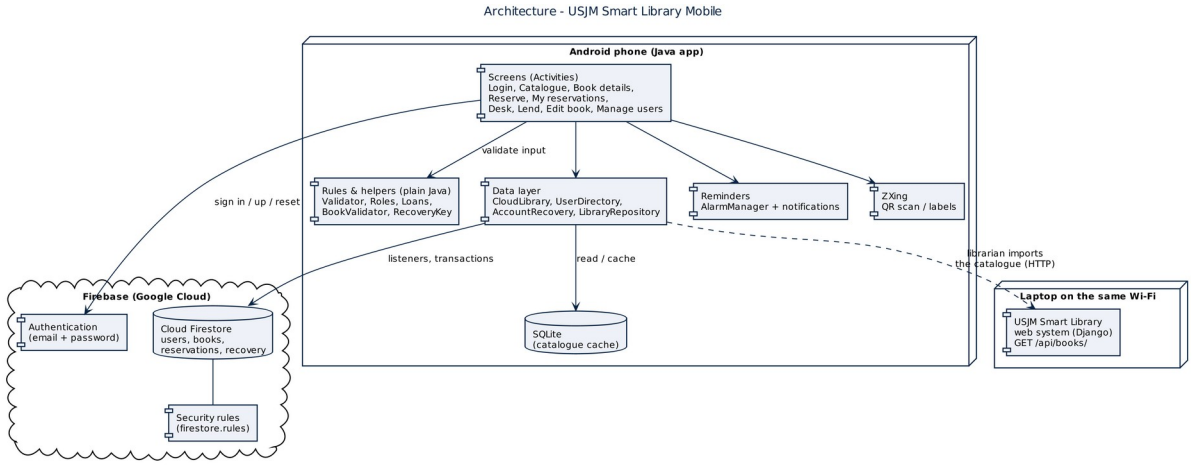


Figure 6: Architecture

Layer	Main classes	Responsibility
Screens (UI)	LoginActivity, SignUpActivity, ForgotPasswordActivity, ResetPasswordActivity, RecoverAccountActivity, MainActivity, BookDetailActivity, ReserveActivity, MyReservationsActivity, DeskActivity, IssueActivity, BookEditActivity, AdminActivity, SetRecoveryPinActivity; adapters	Show data, capture input, navigate with Intents
Rules (plain Java)	Validator, PasswordRules, Roles, Loans, BookValidator, RecoveryKey, QrPayload, ReminderTime, ServerAddress	Validation, password strength, role limits, due dates, recovery keys; covered by 56 unit tests
Data	CloudLibrary, UserDirectory, AccountRecovery, LibraryRepository, Session	Firestore listeners and transactions; the SQLite cache; the signed-in profile
Background	PickupReminders, LoanReminders, BootReceiver, CatalogueSync	Notifications; restoring alarms after a restart; importing from the web system
Storage and services	Firebase Authentication, Cloud Firestore + firestore.rules, SQLite (smartlibrary.db), SharedPreferences	Accounts; shared data protected by rules; offline catalogue; small settings

4.5 Data design

Figure 7 shows the data. Four Firestore collections are shared by all phones; every phone keeps an SQLite copy of the books for fast search and offline browsing.

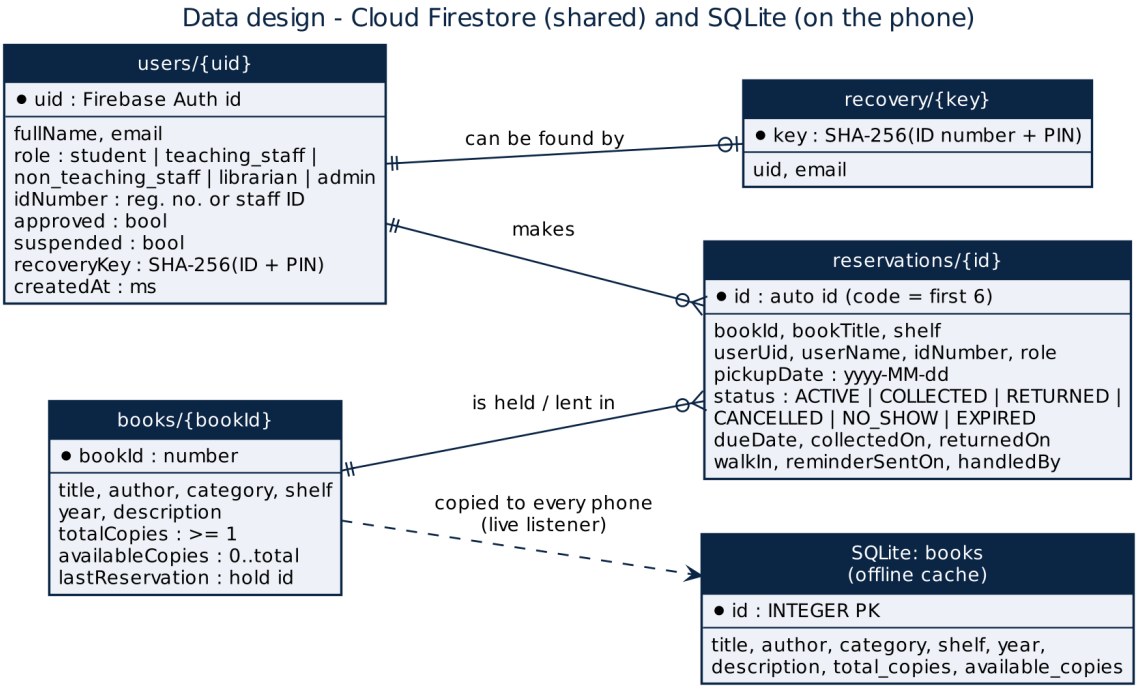


Figure 7: Data design (Cloud Firestore and SQLite)

Reservation status	Meaning	Set by
ACTIVE	Copy held for pickup	Reader (reserve)
COLLECTED	On loan, with a due date	Librarian (Collected or walk-in loan)
RETURNED	Back on the shelf	Librarian
CANCELLED	Reader cancelled the hold	Reader
NO_SHOW	Reader did not come	Librarian
EXPIRED	Pickup day passed without collection	App, automatically

4.6 Wireframes

Figures 8 and 9 are low-fidelity wireframes of the main screens and the navigation between them. The implemented screens follow these layouts, with the USJM navy and gold colours.

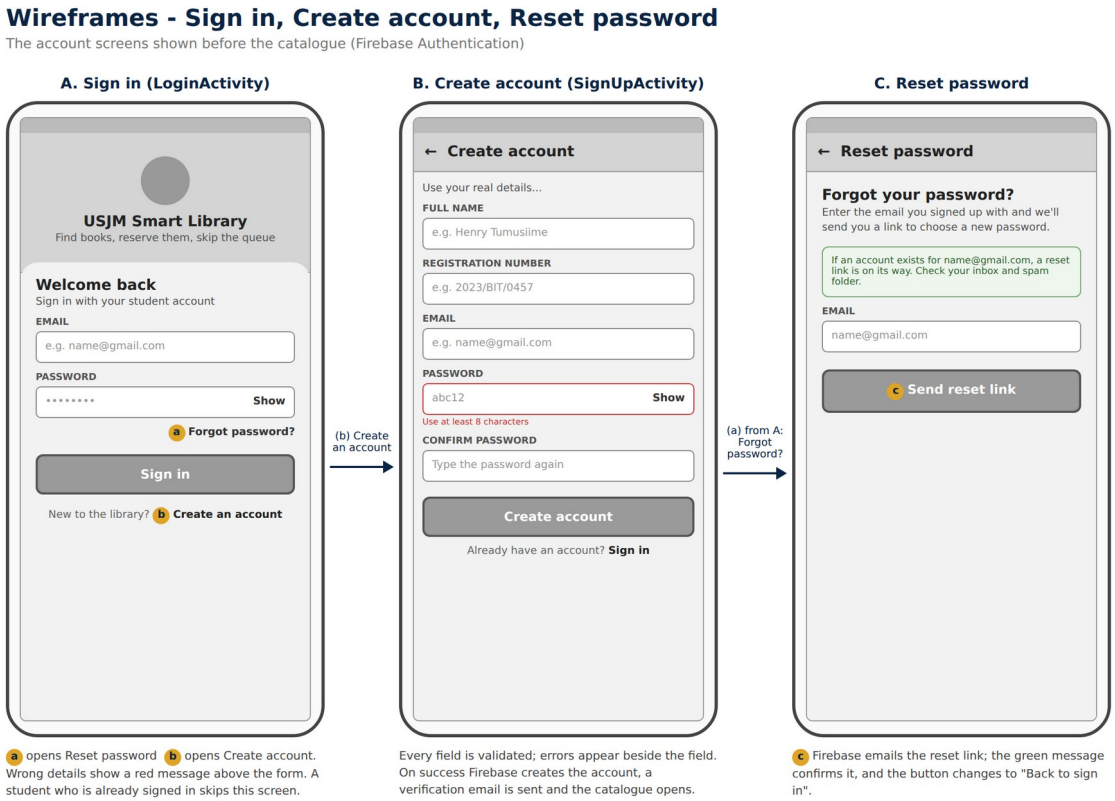


Figure 8a: Account screens. A Sign in, B Create account, C Reset password

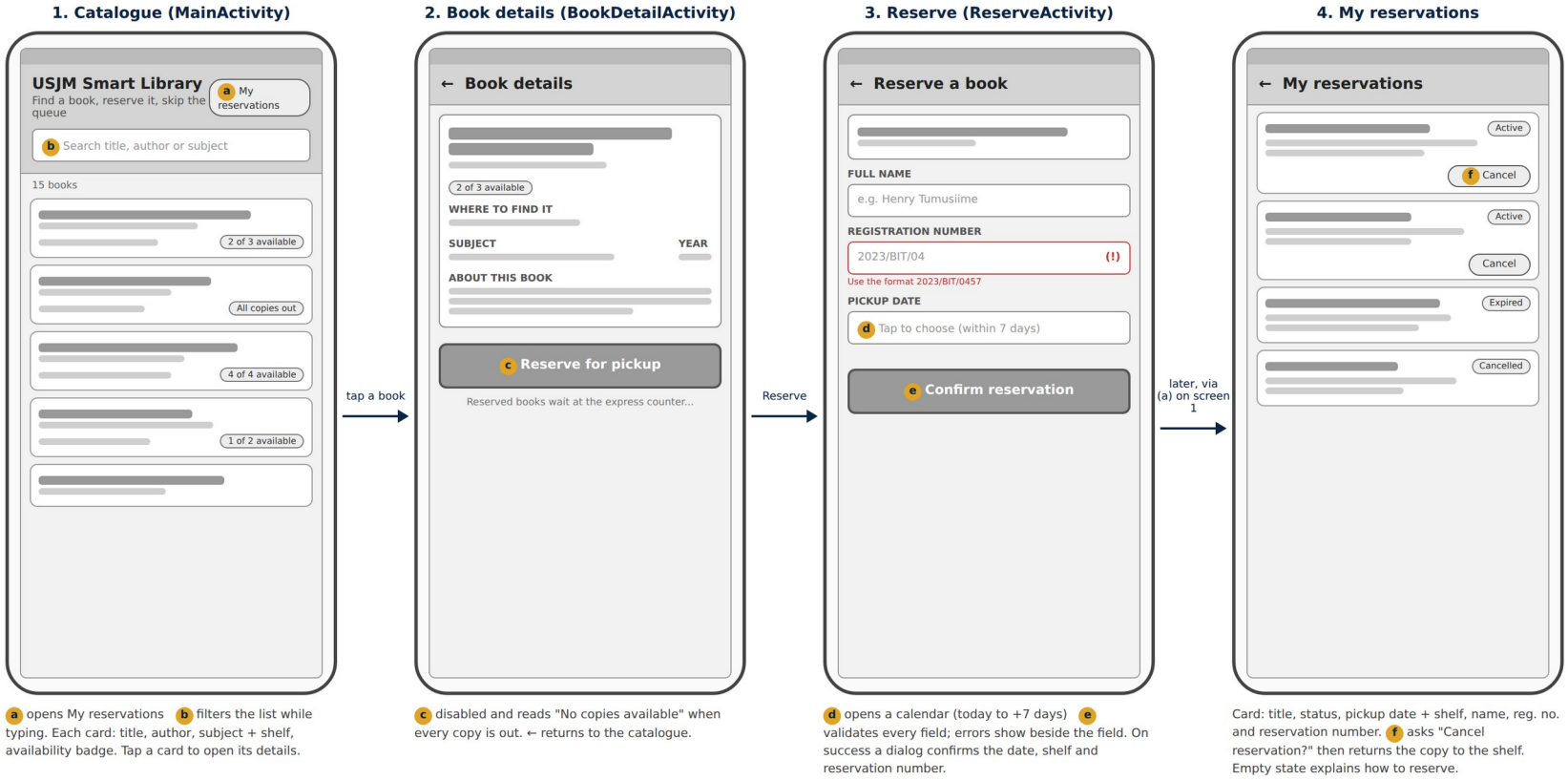


Figure 8b: Reader screens. 1 Catalogue, 2 Book details, 3 Reserve, 4 My reservations

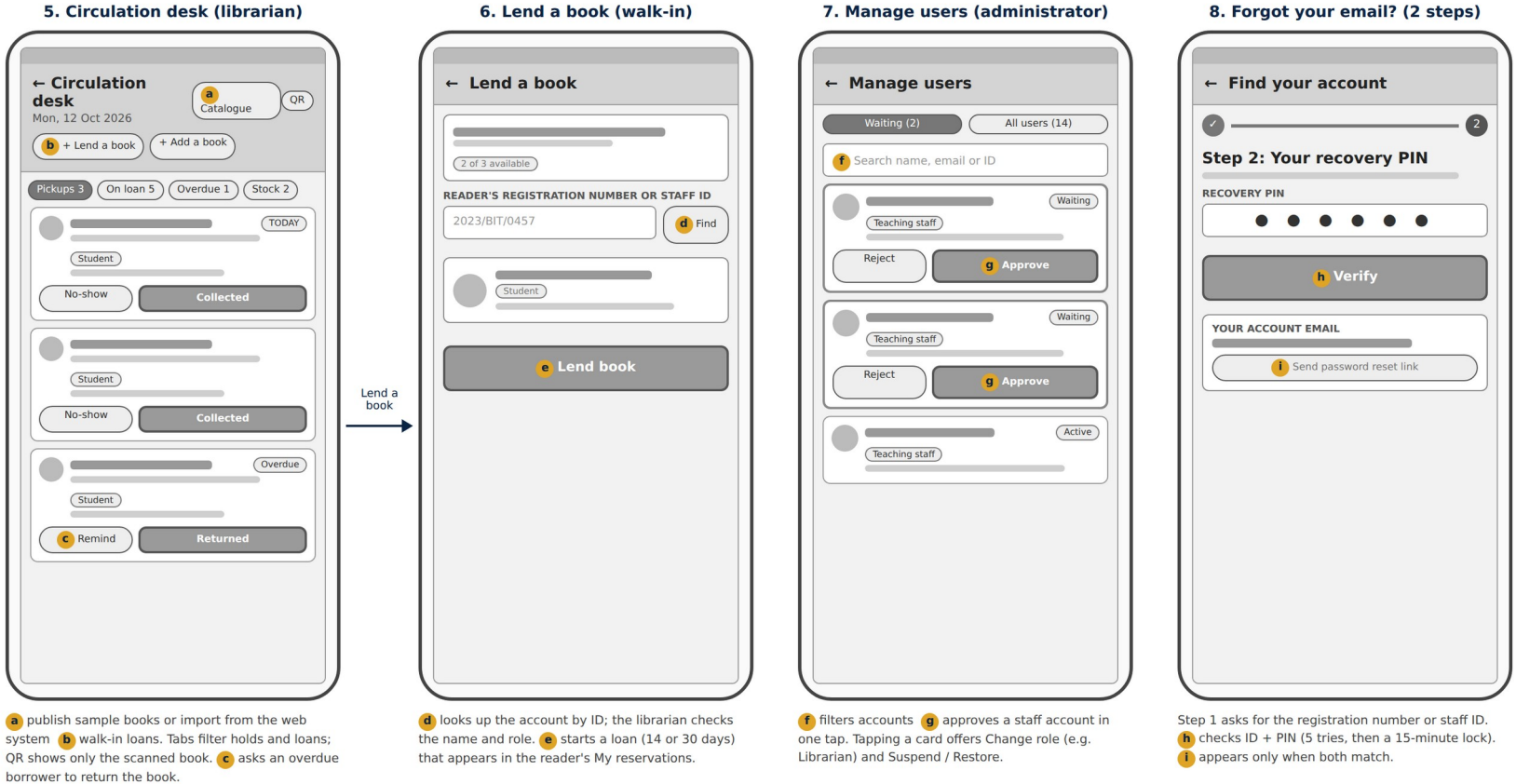


Figure 9: Staff screens and account recovery. 5 Circulation desk, 6 Lend a book, 7 Manage users, 8 Forgot your email?

5. Working Prototype

5.1 Development environment

Item	Choice
IDE and build	Android Studio; Gradle 8.13 with the Android Gradle Plugin 8.13.2
Language	Java 11
Minimum / target SDK	API 24 (Android 7.0) / API 36
User interface	XML layouts with AppCompat; ListView with custom adapters (ViewHolder); dialogs; DatePickerDialog
Accounts	Firebase Authentication (email and password, password-reset email)
Shared data	Cloud Firestore (africa-south1) with live listeners, transactions and security rules
Local data	SQLite through SQLiteOpenHelper (catalogue cache); SharedPreferences
Libraries	ZXing Android Embedded 4.3.0 (QR scanning and labels); Firebase BoM 33.7.0
Background work	AlarmManager, notification channels, a boot receiver
Testing	JUnit 4 (56 local unit tests); manual testing on a Samsung Galaxy S10+ (SM-G975U)
Version control	Git and GitHub
Design tools	PlantUML (UML diagrams); HTML mock-ups (wireframes)

5.2 Screens implemented

Screen	Class	What it does	Who
Sign in	LoginActivity	Email and password with Show/Hide; links to Forgot your email?, Forgot password? and Create account; skipped when signed in	All
Create account	SignUpActivity	Role choice, name, ID number, email, password and confirmation, recovery PIN and confirmation; all validated	All
Reset password	ForgotPasswordActivity	Sends a password-reset link by email	All
New password	ResetPasswordActivity	Opened by the email link (Android App Link): new password and confirmation, live checklist and strength meter; also finishes verify-email links	All
Find your account	RecoverAccountActivity	Two steps: ID number, then recovery PIN; shows the account email and sends a reset link	All
Recovery PIN	SetRecoveryPinActivity	Set or change the 6-digit recovery PIN	All
Catalogue	MainActivity	Greeting and role badge; live list; search; QR scan; tools for the role	All
Book details	BookDetailActivity	Details and availability; Reserve; for librarians Edit, Lend and the QR label	All
Reserve	ReserveActivity	Validated form with a calendar limited by role; saves through a Firestore transaction	Readers

Screen	Class	What it does	Who
My reservations	MyReservationsActivity	Live holds and loans with status and due dates; cancel	Readers
Circulation desk	DeskActivity	Pickups, On loan, Overdue and Stock tabs; Collected, No-show, Returned, Remind; QR filter; catalogue menu	Librarians
Lend a book	IssueActivity	Walk-in loan: find the reader by ID, check the name, lend	Librarians
Add / edit book	BookEditActivity	Validated book form; change copies; remove a book	Librarians
Manage users	AdminActivity	Waiting and All tabs, search; Approve, Reject, Change role, Suspend / Restore	Admins
QR scanner	ZXing CaptureActivity	Reads a book's label to open it (readers) or filter the desk (librarians)	All

Meeting the prototype requirement. The prototype has 17 screens; at least two are required. Navigation uses explicit Intents, and the selected book's id travels as an Intent extra (Listing 3). Working features include forms that save data (create account, reserve, add a book, lend a book), screens that display data from a local source (the catalogue and book details read from SQLite), live shared data (reservations in Cloud Firestore), QR scanning, and notifications.

5.3 Key code

Taking a copy safely. The reader's limits are checked first, then one Firestore transaction reads the book, takes a copy and creates the hold. If two readers reach the last copy at the same moment, only one transaction succeeds (FR13, NFR3).

```
db().runTransaction(t -> {
    DocumentSnapshot b = t.get(bookRef(book.id));
    if (!b.exists()) return "This book is not in the shared catalogue yet.";
    long available = number(b, "availableCopies");
    if (available <= 0) return "All copies of \"" + book.title + "\" are out right now.";
    // take one copy and record which hold took it (checked by the security rules)
    t.update(bookRef(book.id), "availableCopies", available - 1, "updatedAt", now,
             "lastReservation", newRef.getId());
    t.set(newRef, holdFields);          // bookId, userUid, role, pickupDate, status ACTIVE
...
    return null;                        // null = success; a String = the reason it failed
})
```

Listing 1: CloudLibrary.reserve(), the transaction (shortened)

Rules that depend on the role. Limits live in one plain Java class, so they are the same everywhere in the app and easy to unit test. Staff waiting for approval are treated as students.

```
public static int maxActiveReservations(String role, boolean approved) {
    String r = effective(role, approved);      // waiting staff -> student
    if (TEACHING_STAFF.equals(r)) return 10;
    if (STUDENT.equals(r)) return 3;
    return 5;                                   // non-teaching staff, librarians, admins
}
public static int loanDays(String role, boolean approved) {
    return STUDENT.equals(effective(role, approved)) ? 14 : 30;
}
```

Listing 2: Roles.java (shortened)

Navigation between screens. Tapping a book starts the details screen and passes the book's id:

```
Intent i = new Intent(MainActivity.this, BookDetailActivity.class);
i.putExtra(BookDetailActivity.EXTRA_BOOK_ID, id);
startActivity(i);
```

Listing 3: Passing data between screens

Server-side protection. The Firestore security rules repeat the important checks on Google's servers, so even a modified app cannot bypass them. For example, a new account can only be a student or staff member, and staff start unapproved:

```
match /users/{uid} {
  allow create: if signedIn() && request.auth.uid == uid
    && request.resource.data.role in ['student', 'teaching_staff', 'non_teaching_staff']
    && request.resource.data.approved == (request.resource.data.role == 'student')
    && request.resource.data.suspended == false;
  // only administrators may change role, approved or suspended
}
```

Listing 4: firestore.rules (extract)

5.4 Screenshots

Replace each box with a screenshot from the phone (Power + Volume down). Delete this note afterwards.

[Insert screenshot] **Sign in (with "Forgot your email?")**	[Insert screenshot] **Create account showing a validation error**

[Insert screenshot] **Catalogue with the role badge**	[Insert screenshot] **Book details**
[Insert screenshot] **Reserve form with the success dialog**	[Insert screenshot] **My reservations (Active and On loan)**
[Insert screenshot] **Circulation desk: Pickups tab**	[Insert screenshot] **Manage users: Approve / Reject**

Figure 10: Screenshots of the working prototype

6. Version Control

The project is tracked with Git and published on GitHub. Work was committed after each milestone was built and tested on the phone, so the history shows the app growing step by step rather than arriving in one upload.

Repository: https://github.com/HENRIKYVYBZ/2024-BSIT-007-F-Tumusiime-Henry-AndroidExam

Commit	Date	Message	What it added
c3ae5df	4 Oct 2026	Initial project from Android Studio template	Empty Views Activity project (Java)
edc50e6	4 Oct 2026	Catalogue screen: SQLite database, sample books, book list and search	Database, sample books, catalogue list and search
1d6dc4f	4 Oct 2026	Login: sign up, sign in and password reset with Firebase Authentication	Account screens, Validator and its unit tests
ff6aed1	8 Oct 2026	Book details screen and navigation from the catalogue	Book details; Intent navigation
58336e9	8 Oct 2026	Reservation form with validation, My Reservations, cancel and README	Reserve form, My reservations, README
65cd1fb	8 Oct 2026	Add design documents and mid-semester report	Diagrams, wireframes, first report draft
b20d16e	8 Oct 2026	Pickup reminder notifications with AlarmManager	Reminders and the boot receiver
9c6b9bf	8 Oct 2026	QR code scanning to open a book's details (ZXing)	QR scanning and printable labels
00cddd1	8 Oct 2026	Sync the catalogue from the Smart Library web system (REST API)	REST client for the Django web system
e5e736b	8 Oct 2026	User roles: staff sign-up, admin approvals and role-based limits (Firestore)	Roles, Manage users, security rules
317e154	8 Oct 2026	Circulation desk for librarians; shared catalogue and reservations in Firestore	Circulation desk, shared reservations, transactions
hash	date	Desk extras: walk-in lending, add and edit books, return reminders	Lend a book, Add / edit book, return reminders
hash	date	Two-step account recovery with a recovery PIN	Forgot your email?, recovery PIN
hash	date	Confirm the new password when resetting it (App Link + reset page)	New password screen, web reset page

Fill the last three rows after committing (git log --oneline shows the hashes), and insert a screenshot of the Commits page on GitHub here.

7. Testing at the Mid-Semester Stage

7.1 Unit tests

The rules are plain Java, so they are tested on the computer with JUnit 4, using fixed dates so the results never depend on the real day. All 56 tests pass (right-click app/src/test → Run 'Tests in util').

Test class	Tests	What is checked	Result
ValidatorTest	20	Names, registration numbers, emails, passwords and confirmation; pickup dates (past, too far ahead, Sundays, impossible dates)	Pass
RolesTest	8	Only student and staff roles at sign-up; waiting staff get student limits; limits per role; desk and admin rights; staff IDs	Pass
LoansTest	5	Due dates across months and years; overdue days; missed pickups; reservation codes; loan period by role	Pass
BookValidatorTest	4	Book form: required fields, realistic years, copies never below those out; reader ID formats	Pass
PasswordRulesTest	4	Checklist rules for a new password; the confirmation must match exactly; strength meter agrees with the sign-up rules	Pass
RecoveryKeyTest	4	Same ID + PIN give the same key; any change gives a different key; weak PINs refused; email masking	Pass
ReminderTimeTest	4	08:00 reminder time on the pickup day; a reminder 10 seconds later when 08:00 has passed	Pass
ServerAddressTest	4	Web-system address typed with or without http:// and port	Pass
QrPayloadTest	3	Book labels read correctly; other QR codes refused	Pass

7.2 Manual test cases

Run each case on the phone and write Pass or Fail in the last column.

ID	Steps	Expected result	Result
TC1	Create a student account with valid details and a recovery PIN	The catalogue opens with "Hi, <name>" and a Student badge	
TC2	Create an account with the password "abc12" and PIN "123456"	Errors beside both fields; no account is created	
TC3	Sign in with a wrong password	"Incorrect email or password"; stays on Sign in	
TC4	Forgot your email?: enter the reg. number, then the right PIN	The account email is shown; the reset link arrives	
TC4b	Open the reset link on the phone; type two different passwords, then the same strong one	The app opens; Save stays refused until both match; then "Password changed" and sign-in works	
TC5	Forgot your email?: enter a wrong PIN 5 times	"Too many wrong tries" and a 15-minute lock	
TC6	Sign up as Teaching staff	"Teaching staff · pending" badge and banner; student limits	

ID	Steps	Expected result	Result
TC7	As administrator, Approve that account	Banner disappears; the calendar allows 14 days	
TC8	Search "network" in the catalogue	Only matching books remain	
TC9	Scan a book's QR label	That book's details open	
TC10	Reserve a book for today	"Book reserved" with a code; copies drop by 1 on every phone	
TC11	Reserve the same book again	"You already have … reserved or on loan"	
TC12	As a student, try a 4th active reservation	Refused: "at most 3 reservations"	
TC13	As librarian, tap Collected on today's pickup	Moves to On loan; the reader sees "Due back …"	
TC14	As librarian, lend a book to a walk-in reader by reg. number	The loan shows "walk-in" and appears for the reader	
TC15	As librarian, set a book's copies below the number out	Refused with the number of copies out	
TC16	Turn off Wi-Fi and data, then search the catalogue	Search still works from the SQLite copy	

8. Demo Plan (3 to 5 minutes)

Use two phones if possible (one signed in as a student, one as the librarian), or sign out and in between parts.

Time	What to show and say
0:00–0:30	The problem: queues at the desk, no way to hold a book, manual loan tracking. The users and their roles.
0:30–1:15	Accounts: a validation error on Create account; sign in; point out the role badge. Show "Forgot your email?" (ID number, then PIN).
1:15–2:00	Reader: search "java", scan a QR label, open the book, reserve it for today, show the success dialog and My reservations.
2:00–3:00	Librarian: the desk opens; the new hold is under Pickups with TODAY; tap Collected. On the student's phone, My reservations shows "On loan, due back …" live.
3:00–3:40	Librarian: lend a book to a walk-in reader; then Returned. Administrator: approve a waiting lecturer.
3:40–4:30	Design choices: Firestore transactions so a copy is never given twice; security rules; SQLite for offline search; rules in plain Java with 56 passing tests (run them).
4:30–5:00	The Git history on GitHub, and the plan for the final stage.

Demo recording: link to the 3 to 5 minute screen recording

Plan for the final stage

Week	Work
1	Lecturers: book recommendations and course reading lists
2	Administrator statistics; usability and accessibility review against Android design guidelines
3	Full test plan with results; security-rules tests in the Firebase emulator
4	Signed release APK; final report with updated diagrams and code explanations
5	README, declaration of originality, final Git history and submission

9. Tools, Sources and Acknowledgements

- Android Studio, the Android SDK and AndroidX AppCompat (Google); SQLite.
- Firebase Authentication and Cloud Firestore (Google); ZXing Android Embedded (QR codes).
- PlantUML (diagrams); Git and GitHub (version control).
- The sample catalogue lists real published books (titles and authors only), used as test data.
- An AI assistant (Claude, by Anthropic) helped draft parts of the code, diagrams and this report. I reviewed, ran and tested all of it and can explain every part. Check your lecturer's policy on AI tools and keep or adjust this statement.

References

Android Developers. (n.d.). Save data using SQLite. https://developer.android.com/training/data-storage/sqlite

Android Developers. (n.d.). Introduction to activities. https://developer.android.com/guide/components/activities/intro-activities

Android Developers. (n.d.). Create a notification. https://developer.android.com/develop/ui/views/notifications/build-notification

Firebase. (n.d.). Authenticate with Firebase using password-based accounts on Android. https://firebase.google.com/docs/auth/android/password-auth

Firebase. (n.d.). Transactions and batched writes. https://firebase.google.com/docs/firestore/manage-data/transactions

Firebase. (n.d.). Get started with Cloud Firestore Security Rules. https://firebase.google.com/docs/firestore/security/get-started

Fowler, M. (2004). UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.